Open Influence Diagrams: Proofs

Simon Frost

2026-10-16

Contents

Boundary observability 1

Reward kernels: kernels over the expectation semiring 2

The expectation-semiring kernel category 8

Expected utility in context and the contextual identity 12

Decision and utility generators on the typed open-network syntax 14

The fixed-strategy interpretation and its functoriality 17

Optimisation does not compose component by component 23

Boundary observability

```
import CategoricalBayesianNetworksProofs
import InfluenceDiagramsProofs
```

The first module of the proofs of `CategoricalInfluenceDiagrams.jl` (ADR 0018, decision 5). It imports the two proof projects this one builds on: the typed open-network syntax of `CategoricalBayesianNetworks.jl` (ADR 0010, namespace `OpenNet`) and the finite influence-diagram model of `InfluenceDiagrams.jl` (namespace `InfluenceDiagramsProofs`).

It states the one piece of structure that open influence diagrams add to the boundary of an open network: every boundary variable is **observed**, remembered information that a later decision may read, or **hidden**. An interface with observability is an `OpenNet.Interface` with a flag on every port, and the tensor of two such interfaces places ports and flags side by side. **Information locality**, rule 3 of the typed-interface rule, says that no decision of a finite influence diagram reads a variable that its boundary declares hidden.

The later modules prove the rest of ADR 0018, decision 5: the expectation-semiring kernel category with its symmetric monoidal structure (`RewardKernel.lean`, `Category.lean`), the contextual identity (`Context.lean`), decision and utility generators on the typed syntax (`Syntax.lean`), the functoriality of the fixed-strategy interpretation (`Interpretation.lean`) and the isolated-optimisation and hidden-input obstructions (`Obstruction.lean`). The observability flags of this module are not read by that semantics.

```
set_option autoImplicit false

namespace OpenInfluence

/-- The observability of a boundary variable (ADR 0018, decision 2). -/
inductive Observability where
  /-- Remembered information that a later decision may read. -/
  | observed
  /-- State that crosses the boundary but that no decision on the far side may read. -/
  | hidden
  deriving DecidableEq, Repr
```

```
/-- An interface of the open-network syntax with an observability flag on every port. -/
structure ObservedInterface (S : OpenNet.Signature) where
  /-- The underlying finite typed interface. -/
  iface : OpenNet.Interface S
  /-- The flag of each port. -/
  flag : iface.Port → Observability

namespace ObservedInterface

variable {S : OpenNet.Signature}

/-- The tensor of two interfaces with observability: ports and flags side by side. -/
def sum (X Y : ObservedInterface S) : ObservedInterface S :=
  ⟨X.iface.sum Y.iface, Sum.elim X.flag Y.flag⟩

/-- The empty interface, the unit of `sum`. -/
def empty (S : OpenNet.Signature) : ObservedInterface S :=
  ⟨OpenNet.Interface.empty S, Empty.elim⟩

/-- The tensor keeps the flags of the left interface. -/
theorem sum_flag_inl (X Y : ObservedInterface S) (p : X.iface.Port) :
    (X.sum Y).flag (Sum.inl p) = X.flag p := rfl

/-- The tensor keeps the flags of the right interface. -/
theorem sum_flag_inr (X Y : ObservedInterface S) (p : Y.iface.Port) :
    (X.sum Y).flag (Sum.inr p) = Y.flag p := rfl

end ObservedInterface

open InfluenceDiagramsProofs

/-- Information locality (rule 3 of ADR 0018, decision 2): given the observability of the
boundary variables of a finite influence diagram (`none` for an internal variable), no
decision has a hidden boundary variable in its information set. -/
def InformationLocal (D : FinInfluenceDiagram) (boundary : D.V → Option Observability) :
    Prop :=
  ∀ d v, v ∈ D.info d → boundary v ≠ some .hidden

/-- A diagram whose boundary declares nothing hidden is information-local. -/
theorem informationLocal_of_no_hidden (D : FinInfluenceDiagram)
    (boundary : D.V → Option Observability) (h : ∀ v, boundary v ≠ some .hidden) :
    InformationLocal D boundary :=
  fun _ v _ ⇒ h v

/-- Revealing boundary variables (turning hidden flags into observed or internal ones)
preserves information locality. -/
theorem InformationLocal.of_reveal {D : FinInfluenceDiagram}
    {b b' : D.V → Option Observability}
    (h : InformationLocal D b) (hb : ∀ v, b' v = some .hidden → b v = some .hidden) :
    InformationLocal D b' :=
  fun d v hv hid ⇒ h d v hv (hb v hid)

end OpenInfluence
```

Reward kernels: kernels over the expectation semiring

```
import FiniteKernelsProofs.Finite.Kernel
import Mathlib.Tactic.Abel
```

A **reward kernel** `X → Y` between finite types is a pair `(K, W)` of real tables. `K x y` is the probability of reaching boundary state `y` from `x`, and `W x y` is the probability-weighted reward earned on the way (ADR 0018, decision 3). Composition, identity and tensor are those of `CategoricalInfluenceDiagrams.jl`'s `RewardKernel` (`src/semantics.jl`):

- $(K_1, W_1) ; (K_2, W_2) = (K_1K_2, W_1K_2 + K_1W_2)$;
- $\mathrm{id} = (\mathrm{id}, 0)$;
- $(K_1, W_1) \otimes (K_2, W_2) = (K_1 \otimes K_2, W_1 \otimes K_2 + K_1 \otimes W_2)$.

This is kernel composition over the expectation semiring $(p, r)\cdot(q, s) = (pq, ps + rq)$. `K` reuses `FiniteKernelsProofs.Finite.Kernel` unchanged; only the reward component is new.

This module proves, for arbitrary real tables (no nonnegativity or normalisation is needed for any of them):

- the category laws: `id_comp`, `comp_id` and `comp_assoc`;
- the functoriality of the tensor: `tensor_comp_tensor` (the interchange law `(f ⊗ g) ; (f' ⊗ g') = (f ; f') ⊗ (g ; g')`) and `tensor_id`;
- the naturality of the structural maps, all lifted from `FiniteKernels` with zero reward: `tensor_assoc` (associator), `tensor_unit_left` and `tensor_unit_right` (unitors) and `tensor_swap` (symmetry);
- that copy and discard lifted with zero reward satisfy the comonoid laws, and that stochasticity of `K` is preserved by composition and tensor (`Stochastic.comp`, `Stochastic.tensor`).

Discard is **not** natural: `comp_discard` computes `f ; discard = (K ; discard, row sums of W)`, and `Examples.discard_not_natural` exhibits a stochastic reward kernel whose composite with discard is not discard. The Mathlib packaging of these laws as a symmetric monoidal category is `Category.lean`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite
```

Bilinearity of kernel composition and tensor

```
namespace KernelAlgebra

variable {X Y Z X₁ X₂ Y₁ Y₂ : Type}

theorem kcomp_add_left [Fintype Y] (k k' : Kernel X Y) (l : Kernel Y Z) :
    Kernel.comp (k + k') l = Kernel.comp k l + Kernel.comp k' l := by
  funext x z
  simp [Kernel.comp, add_mul, Finset.sum_add_distrib]

theorem kcomp_add_right [Fintype Y] (k : Kernel X Y) (l l' : Kernel Y Z) :
    Kernel.comp k (l + l') = Kernel.comp k l + Kernel.comp k l' := by
  funext x z
  simp [Kernel.comp, mul_add, Finset.sum_add_distrib]

theorem kcomp_zero_left [Fintype Y] (l : Kernel Y Z) :
    Kernel.comp (0 : Kernel X Y) l = 0 := by
  funext x z
  simp [Kernel.comp]

theorem kcomp_zero_right [Fintype Y] (k : Kernel X Y) :
    Kernel.comp k (0 : Kernel Y Z) = 0 := by
  funext x z
  simp [Kernel.comp]

theorem ktensor_add_left (k k' : Kernel X₁ Y₁) (l : Kernel X₂ Y₂) :
    Kernel.tensor (k + k') l = Kernel.tensor k l + Kernel.tensor k' l := by
  funext x y
  simp [Kernel.tensor, add_mul]

theorem ktensor_add_right (k : Kernel X₁ Y₁) (l l' : Kernel X₂ Y₂) :
    Kernel.tensor k (l + l') = Kernel.tensor k l + Kernel.tensor k l' := by
  funext x y
  simp [Kernel.tensor, mul_add]
```

```
theorem ktensor_zero_left (l : Kernel X₂ Y₂) :
    Kernel.tensor (0 : Kernel X₁ Y₁) l = 0 := by
  funext x y
  simp [Kernel.tensor]

theorem ktensor_zero_right (k : Kernel X₁ Y₁) :
    Kernel.tensor k (0 : Kernel X₂ Y₂) = 0 := by
  funext x y
  simp [Kernel.tensor]

end KernelAlgebra

open KernelAlgebra

/-- A reward kernel `X → Y`: a probability table `K` and a probability-weighted reward
table `W` of the same shape (Julia `RewardKernel(K, W)`). -/
@[ext] structure RewardKernel (X Y : Type) where
  /-- `K x y`: the probability of reaching `y` from `x`. -/
  K : Kernel X Y
  /-- `W x y`: the reward earned on the way from `x` to `y`, weighted by its probability. -/
  W : Kernel X Y

namespace RewardKernel

variable {X Y Z V X₁ X₂ X₃ Y₁ Y₂ Y₃ Z₁ Z₂ : Type}
```

Constructions

```
/-- Serial composition `(K₁, W₁) ; (K₂, W₂) = (K₁K₂, W₁K₂ + K₁W₂)` (diagrammatic order). -/
def comp [Fintype Y] (f : RewardKernel X Y) (g : RewardKernel Y Z) : RewardKernel X Z :=
  ⟨Kernel.comp f.K g.K, Kernel.comp f.W g.K + Kernel.comp f.K g.W⟩

/-- A kernel with zero reward. Identities and all structural maps are of this form. -/
def lift (k : Kernel X Y) : RewardKernel X Y := ⟨k, 0⟩

/-- The identity `(id, 0)`. -/
def id (X : Type) [DecidableEq X] : RewardKernel X X := lift (Kernel.idK X)

/-- The tensor `(K₁ ⊗ K₂, W₁ ⊗ K₂ + K₁ ⊗ W₂)`. -/
def tensor (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂) :
    RewardKernel (X₁ × X₂) (Y₁ × Y₂) :=
  ⟨Kernel.tensor f.K g.K, Kernel.tensor f.W g.K + Kernel.tensor f.K g.W⟩

/-- The deterministic reward kernel of a function, with zero reward. -/
def ofFun [DecidableEq Y] (e : X → Y) : RewardKernel X Y := lift (Kernel.ofFun e)

/-- Copy, lifted with zero reward. -/
def copy (X : Type) [DecidableEq X] : RewardKernel X (X × X) := lift (Kernel.copy X)

/-- Discard, lifted with zero reward. -/
def discard (X : Type) : RewardKernel X Unit := lift (Kernel.discard X)

/-- The probability component is stochastic (nonnegative and normalised); the reward
component is unconstrained. -/
def Stochastic [Fintype Y] (f : RewardKernel X Y) : Prop := Kernel.Stochastic f.K

@[simp] theorem comp_K [Fintype Y] (f : RewardKernel X Y) (g : RewardKernel Y Z) :
    (f.comp g).K = Kernel.comp f.K g.K := rfl

@[simp] theorem comp_W [Fintype Y] (f : RewardKernel X Y) (g : RewardKernel Y Z) :
    (f.comp g).W = Kernel.comp f.W g.K + Kernel.comp f.K g.W := rfl

@[simp] theorem tensor_K (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂) :
    (f.tensor g).K = Kernel.tensor f.K g.K := rfl
```

```
@[simp] theorem tensor_W (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂) :
    (f.tensor g).W = Kernel.tensor f.W g.K + Kernel.tensor f.K g.W := rfl

@[simp] theorem lift_K (k : Kernel X Y) : (lift k).K = k := rfl
@[simp] theorem lift_W (k : Kernel X Y) : (lift k).W = 0 := rfl
```

Category laws

```
theorem id_comp [Fintype X] [DecidableEq X] (f : RewardKernel X Y) : (id X).comp f = f := by
  ext1
  · exact Kernel.idK_comp f.K
  · change Kernel.comp 0 f.K + Kernel.comp (Kernel.idK X) f.W = f.W
    rw [kcomp_zero_left, Kernel.idK_comp, zero_add]

theorem comp_id [Fintype Y] [DecidableEq Y] (f : RewardKernel X Y) : f.comp (id Y) = f := by
  ext1
  · exact Kernel.comp_idK f.K
  · change Kernel.comp f.W (Kernel.idK Y) + Kernel.comp f.K 0 = f.W
    rw [kcomp_zero_right, Kernel.comp_idK, add_zero]

theorem comp_assoc [Fintype Y] [Fintype Z] (f : RewardKernel X Y) (g : RewardKernel Y Z)
    (h : RewardKernel Z V) : (f.comp g).comp h = f.comp (g.comp h) := by
  ext1
  · exact Kernel.comp_assoc f.K g.K h.K
  · simp only [comp_W, comp_K, kcomp_add_left, kcomp_add_right, Kernel.comp_assoc]
    abel
```

Lifted kernels

```
theorem lift_comp_lift [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) :
    (lift k).comp (lift l) = lift (Kernel.comp k l) := by
  ext1
  · rfl
  · simp [kcomp_zero_left, kcomp_zero_right]

theorem lift_tensor_lift (k : Kernel X₁ Y₁) (l : Kernel X₂ Y₂) :
    (lift k).tensor (lift l) = lift (Kernel.tensor k l) := by
  ext1
  · rfl
  · simp [ktensor_zero_left, ktensor_zero_right]
```

Functoriality of the tensor

```
/-- The interchange law `(f ⊗ g) ; (f' ⊗ g') = (f ; f') ⊗ (g ; g')`. -/
theorem tensor_comp_tensor [Fintype Y₁] [Fintype Y₂] (f : RewardKernel X₁ Y₁)
    (g : RewardKernel X₂ Y₂) (f' : RewardKernel Y₁ Z₁) (g' : RewardKernel Y₂ Z₂) :
    (f.tensor g).comp (f'.tensor g') = (f.comp f').tensor (g.comp g') := by
  ext1
  · exact Kernel.comp_tensor f.K g.K f'.K g'.K
  · simp only [comp_W, comp_K, tensor_W, tensor_K, kcomp_add_left, kcomp_add_right,
      ktensor_add_left, ktensor_add_right, Kernel.comp_tensor]
    abel

/-- The tensor of identities is the identity. -/
theorem tensor_id [DecidableEq X] [DecidableEq Y] :
    (id X).tensor (id Y) = id (X × Y) := by
  simp only [id, lift_tensor_lift, Kernel.tensor_idK]
```

Naturality of the structural maps

```
/-- Associativity of the tensor up to the associator (lifted with zero reward). -/
theorem tensor_assoc [Fintype X₁] [Fintype X₂] [Fintype X₃] [Fintype Y₁] [Fintype Y₂]
    [Fintype Y₃] [DecidableEq X₁] [DecidableEq X₂] [DecidableEq X₃] [DecidableEq Y₁]
    [DecidableEq Y₂] [DecidableEq Y₃] (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂)
    (h : RewardKernel X₃ Y₃) :
    ((f.tensor g).tensor h).comp (lift (Kernel.assoc Y₁ Y₂ Y₃)) =
      (lift (Kernel.assoc X₁ X₂ X₃)).comp (f.tensor (g.tensor h)) := by
  ext1
  · exact Kernel.tensor_assoc f.K g.K h.K
  · simp only [comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_add_right, kcomp_zero_left, kcomp_zero_right, ktensor_add_left, ktensor_add_right,
      Kernel.tensor_assoc]
    abel

/-- Left unit of the tensor up to the left unitor. -/
theorem tensor_unit_left [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
    (f : RewardKernel X Y) :
    ((id Unit).tensor f).comp (lift (Kernel.leftUnitor Y)) =
      (lift (Kernel.leftUnitor X)).comp f := by
  ext1
  · exact Kernel.tensor_unit_left f.K
  · simp only [id, comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_zero_left, kcomp_zero_right, ktensor_zero_left, Kernel.tensor_unit_left]
    simp

/-- Right unit of the tensor up to the right unitor. -/
theorem tensor_unit_right [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
    (f : RewardKernel X Y) :
    (f.tensor (id Unit)).comp (lift (Kernel.rightUnitor Y)) =
      (lift (Kernel.rightUnitor X)).comp f := by
  ext1
  · exact Kernel.tensor_unit_right f.K
  · simp only [id, comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_zero_left, kcomp_zero_right, ktensor_zero_right, Kernel.tensor_unit_right]
    simp

/-- Naturality of the symmetry. -/
theorem tensor_swap [Fintype X₁] [Fintype X₂] [Fintype Y₁] [Fintype Y₂] [DecidableEq X₁]
    [DecidableEq X₂] [DecidableEq Y₁] [DecidableEq Y₂] (f : RewardKernel X₁ Y₁)
    (g : RewardKernel X₂ Y₂) :
    (f.tensor g).comp (lift (Kernel.swap Y₁ Y₂)) =
      (lift (Kernel.swap X₁ X₂)).comp (g.tensor f) := by
  ext1
  · exact Kernel.tensor_swap f.K g.K
  · simp only [comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_add_right, kcomp_zero_left, kcomp_zero_right, Kernel.tensor_swap]
    abel
```

Copy and discard with zero reward

```
/-- Coassociativity of the lifted copy. -/
theorem copy_assoc [Fintype X] [DecidableEq X] :
    (copy X).comp ((id X).tensor (copy X)) =
      ((copy X).comp ((copy X).tensor (id X))).comp (lift (Kernel.assoc X X X)) := by
  simp only [copy, id, lift_tensor_lift, lift_comp_lift, Kernel.copy_assoc]

/-- Left counit law of the lifted copy and discard. -/
theorem copy_discard_left [Fintype X] [DecidableEq X] :
    (copy X).comp ((discard X).tensor (id X)) = lift (Kernel.leftUnitorInv X) := by
  simp only [copy, discard, id, lift_tensor_lift, lift_comp_lift, Kernel.copy_discard_left]

/-- Right counit law of the lifted copy and discard. -/
theorem copy_discard_right [Fintype X] [DecidableEq X] :
```

```
    (copy X).comp ((id X).tensor (discard X)) = lift (Kernel.rightUnitorInv X) := by
  simp only [copy, discard, id, lift_tensor_lift, lift_comp_lift, Kernel.copy_discard_right]

/-- Cocommutativity of the lifted copy. -/
theorem copy_swap [Fintype X] [DecidableEq X] :
    (copy X).comp (lift (Kernel.swap X X)) = copy X := by
  simp only [copy, lift_comp_lift, Kernel.copy_swap]

/-- Composing with discard keeps the discarded probability and sums the reward over the
output: `f ; discard = (K ; discard, x ↦ Σ_y W x y)`. -/
theorem comp_discard [Fintype Y] (f : RewardKernel X Y) :
    f.comp (discard Y) = ⟨Kernel.comp f.K (Kernel.discard Y), fun x _ ⇒ ∑ y, f.W x y⟩ := by
  ext1
  · rfl
  · funext x u
    simp [comp, discard, lift, Kernel.comp]

/-- Discard is natural exactly for the reward kernels with normalised `K` and zero total
reward from every input. In particular it is not natural in general. -/
theorem comp_discard_eq_discard_iff [Fintype Y] (f : RewardKernel X Y) :
    f.comp (discard Y) = discard X ↔ Kernel.Normalised f.K ∧ ∀ x, ∑ y, f.W x y = 0 := by
  rw [comp_discard]
  constructor
  · intro h
    have hK := congrArg RewardKernel.K h
    have hW := congrArg RewardKernel.W h
    refine ⟨fun x ⇒ ?_, fun x ⇒ ?_⟩
    · have := congrFun (congrFun hK x) ()
      simpa [Kernel.comp, discard, lift] using this
    · exact congrFun (congrFun hW x) ()
  · rintro ⟨hK, hW⟩
    ext1
    · funext x u
      simpa [Kernel.comp, discard, lift] using hK x
    · funext x u
      exact hW x
```

Stochasticity

```
theorem Stochastic.comp [Fintype Y] [Fintype Z] {f : RewardKernel X Y}
    {g : RewardKernel Y Z} (hf : f.Stochastic) (hg : g.Stochastic) : (f.comp g).Stochastic :=
  Kernel.Stochastic.comp hf hg

theorem Stochastic.tensor [Fintype Y₁] [Fintype Y₂] {f : RewardKernel X₁ Y₁}
    {g : RewardKernel X₂ Y₂} (hf : f.Stochastic) (hg : g.Stochastic) :
    (f.tensor g).Stochastic :=
  Kernel.Stochastic.tensor hf hg

theorem Stochastic.ofFun [Fintype Y] [DecidableEq Y] (e : X → Y) : (ofFun e).Stochastic :=
  Kernel.Stochastic.ofFun e

theorem Stochastic.id [Fintype X] [DecidableEq X] : (id X).Stochastic :=
  Kernel.Stochastic.ofFun _

theorem Stochastic.discard : (discard X).Stochastic :=
  ⟨Kernel.Nonneg.discard, Kernel.Normalised.discard⟩

end RewardKernel
```

Discard is not natural

```
namespace RewardKernel.Examples
```

```
/-- A one-point process that is certain to stay put and earns reward one. -/
def unitReward : RewardKernel Unit Unit := ⟨Kernel.idK Unit, fun _ _ ⇒ 1⟩

theorem unitReward_stochastic : unitReward.Stochastic :=
  Kernel.Stochastic.ofFun (fun u : Unit ⇒ u)

/-- Discard, lifted with zero reward, is not natural: a stochastic reward kernel that earns
reward one, followed by discard, still carries that reward, so it is not discard (as ADR 0010
recorded `discard_not_natural` for the open-network syntax). -/
theorem discard_not_natural :
    unitReward.Stochastic ∧ unitReward.comp (discard Unit) ≠ discard Unit := by
  refine ⟨unitReward_stochastic, fun h ⇒ ?_⟩
  have hW := congrFun (congrFun (congrArg RewardKernel.W h) ()) ()
  rw [comp_discard] at hW
  simp [unitReward, discard, lift] at hW

end RewardKernel.Examples

end OpenInfluence
```

The expectation-semiring kernel category

```
import CategoricalInfluenceDiagramsProofs.RewardKernel
import FiniteKernelsProofs.Theory.FinStoch
import Mathlib.CategoryTheory.CopyDiscardCategory.Basic
```

`FinRew` packages the laws of `RewardKernel.lean` as Mathlib instances. Objects are finite types with decidable equality (the empty type included), as in `FinStoch`. A morphism is a reward kernel whose probability component `K` is stochastic; the reward component `W` is an arbitrary real table, of either sign. Composition, identity and tensor are those of `RewardKernel`; the associator, unitors and symmetry are the deterministic reindexings of `FinStoch`, lifted with zero reward.

The instances are a `Category`, a `MonoidalCategory`, a `SymmetricCategory` and a `CopyDiscardCategory` (every object a commutative comonoid, copy and discard lifted with zero reward, coherent with the tensor). There is deliberately no `MarkovCategory` instance: discard is not natural, because `f ≫ ε` keeps the expected reward of `f` (`FinRew.discard_not_natural`).

`FinRew.lift` is the faithful embedding of `FinStoch` with zero reward; it preserves composition, identities and tensor, so `FinStoch` is the reward-free part of `FinRew`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open CategoryTheory MonoidalCategory FiniteKernelsProofs FiniteKernelsProofs.Finite

/-- Finite state types with decidable equality, including the empty type. -/
structure FinRew where
  /-- The finite set of boundary states. -/
  carrier : Type
  [fintype : Fintype carrier]
  [decEq : DecidableEq carrier]

attribute [instance] FinRew.fintype FinRew.decEq

namespace FinRew

instance instCategory : Category FinRew where
  Hom X Y := {r : RewardKernel X.carrier Y.carrier // r.Stochastic}
  id X := ⟨RewardKernel.id X.carrier, RewardKernel.Stochastic.id⟩
  comp f g := ⟨f.1.comp g.1, f.2.comp g.2⟩
```

```
  id_comp f := Subtype.ext (RewardKernel.id_comp f.1)
  comp_id f := Subtype.ext (RewardKernel.comp_id f.1)
  assoc f g h := Subtype.ext (RewardKernel.comp_assoc f.1 g.1 h.1)

variable {X Y Z : FinRew}

@[simp] theorem comp_val (f : X ⟶ Y) (g : Y ⟶ Z) : (f ≫ g).val = f.val.comp g.val := rfl

@[simp] theorem id_val (X : FinRew) : (𝟙 X : X ⟶ X).val = RewardKernel.id X.carrier := rfl

/-- A stochastic kernel with zero reward. -/
def lift (k : Kernel X.carrier Y.carrier) (hk : Kernel.Stochastic k) : X ⟶ Y :=
  ⟨RewardKernel.lift k, hk⟩

/-- The deterministic morphism of a function, with zero reward. -/
def det (f : X.carrier → Y.carrier) : X ⟶ Y := lift (Kernel.ofFun f) (Kernel.Stochastic.ofFun f)

/-- A state-space equivalence induces an isomorphism with zero reward. -/
def isoOfEquiv (e : X.carrier ≃ Y.carrier) : X ≅ Y where
  hom := det e
  inv := det e.symm
  hom_inv_id := Subtype.ext (by
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    rw [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    exact congrArg RewardKernel.lift (Kernel.ofFun_congr fun x ⇒ e.symm_apply_apply x))
  inv_hom_id := Subtype.ext (by
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    rw [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    exact congrArg RewardKernel.lift (Kernel.ofFun_congr fun y ⇒ e.apply_symm_apply y))

instance instMonoidalCategoryStruct : MonoidalCategoryStruct FinRew where
  tensorObj X Y := ⟨X.carrier × Y.carrier⟩
  tensorUnit := ⟨Unit⟩
  tensorHom f g := ⟨f.1.tensor g.1, f.2.tensor g.2⟩
  whiskerLeft X _ _ g := ⟨(RewardKernel.id X.carrier).tensor g.1,
    RewardKernel.Stochastic.id.tensor g.2⟩
  whiskerRight f Y := ⟨f.1.tensor (RewardKernel.id Y.carrier),
    f.2.tensor RewardKernel.Stochastic.id⟩
  associator X Y Z := isoOfEquiv (Equiv.prodAssoc X.carrier Y.carrier Z.carrier)
  leftUnitor X := isoOfEquiv (Equiv.punitProd X.carrier)
  rightUnitor X := isoOfEquiv (Equiv.prodPUnit X.carrier)

@[simp] theorem tensorHom_val {X Y X' Y' : FinRew} (f : X ⟶ Y) (g : X' ⟶ Y') :
    (f ⊗ₘ g).val = f.val.tensor g.val := rfl

instance instMonoidalCategory : MonoidalCategory FinRew :=
  MonoidalCategory.ofTensorHom
    (id_tensorHom_id := fun _ _ ⇒ Subtype.ext RewardKernel.tensor_id)
    (id_tensorHom := by intros; rfl)
    (tensorHom_id := by intros; rfl)
    (tensorHom_comp_tensorHom := fun f g f' g' ⇒
      Subtype.ext (RewardKernel.tensor_comp_tensor f.1 g.1 f'.1 g'.1))
    (associator_naturality := fun f g h ⇒ Subtype.ext (RewardKernel.tensor_assoc f.1 g.1 h.1))
    (leftUnitor_naturality := fun f ⇒ Subtype.ext (RewardKernel.tensor_unit_left f.1))
    (rightUnitor_naturality := fun f ⇒ Subtype.ext (RewardKernel.tensor_unit_right f.1))
    (pentagon := by
      intro W X Y Z
      apply Subtype.ext
      change ((RewardKernel.lift _).tensor (RewardKernel.id _)).comp
        ((RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _))) =
          (RewardKernel.lift _).comp (RewardKernel.lift _)
      simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
        Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
      rfl)
    (triangle := by
      intro X Y
```

```
      apply Subtype.ext
      change (RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)) =
          (RewardKernel.lift _).tensor (RewardKernel.id _)
      simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
        Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
      rfl)

instance instSymmetricCategory : SymmetricCategory FinRew where
  braiding X Y := isoOfEquiv (Equiv.prodComm X.carrier Y.carrier)
  braiding_naturality_left := by
    intro X Y f Z
    exact Subtype.ext (RewardKernel.tensor_swap f.1 (RewardKernel.id _))
  braiding_naturality_right := by
    intro X Y Z f
    exact Subtype.ext (RewardKernel.tensor_swap (RewardKernel.id _) f.1)
  hexagon_forward := by
    intro X Y Z
    apply Subtype.ext
    change (RewardKernel.lift _).comp ((RewardKernel.lift _).comp (RewardKernel.lift _)) =
      ((RewardKernel.lift _).tensor (RewardKernel.id _)).comp
        ((RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)))
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
      Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl
  hexagon_reverse := by
    intro X Y Z
    apply Subtype.ext
    change (RewardKernel.lift _).comp ((RewardKernel.lift _).comp (RewardKernel.lift _)) =
      ((RewardKernel.id _).tensor (RewardKernel.lift _)).comp
        ((RewardKernel.lift _).comp ((RewardKernel.lift _).tensor (RewardKernel.id _)))
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
      Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl
  symmetry := by
    intro X Y
    apply Subtype.ext
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    simp only [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    rfl

/-- Copy and discard, lifted with zero reward. -/
instance instComonObj (X : FinRew) : ComonObj X where
  counit := lift (Kernel.discard X.carrier) ⟨Kernel.Nonneg.discard, Kernel.Normalised.discard⟩
  comul := det fun x ⇒ (x, x)
  counit_comul := Subtype.ext (by
    change (RewardKernel.lift _).comp ((RewardKernel.lift _).tensor (RewardKernel.id _)) =
      RewardKernel.lift _
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.copy_discard_left)
  comul_counit := Subtype.ext (by
    change (RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)) =
      RewardKernel.lift _
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.copy_discard_right)
  comul_assoc := Subtype.ext (by
    change (RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)) =
      (RewardKernel.lift _).comp
        (((RewardKernel.lift _).tensor (RewardKernel.id _)).comp (RewardKernel.lift _))
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
      Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl)

instance instIsCommComonObj (X : FinRew) : IsCommComonObj X where
  comul_comm := Subtype.ext (by
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    simp only [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
```

```
    rfl)

open scoped ComonObj

/-- The structural `tensorμ` of Mathlib is the middle-swap reindexing with zero reward. -/
theorem tensorμ_val (X Y : FinRew) :
    (MonoidalCategory.tensorμ X X Y Y).val =
      RewardKernel.lift (Kernel.tensorμ X.carrier Y.carrier) := by
  simp only [MonoidalCategory.tensorμ]
  change (RewardKernel.lift _).comp
    (((RewardKernel.id _).tensor (RewardKernel.lift _)).comp
      (((RewardKernel.id _).tensor ((RewardKernel.lift _).tensor (RewardKernel.id _))).comp
        (((RewardKernel.id _).tensor (RewardKernel.lift _)).comp (RewardKernel.lift _)))) = _
  simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
    Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun, Kernel.tensorμ]
  rfl

instance instCopyDiscardCategory : CopyDiscardCategory FinRew where
  copy_tensor X Y := Subtype.ext (by
    change RewardKernel.lift (Kernel.copy (X.carrier × Y.carrier)) =
      ((RewardKernel.lift (Kernel.copy X.carrier)).tensor
        (RewardKernel.lift (Kernel.copy Y.carrier))).comp
        (MonoidalCategory.tensorμ X X Y Y).val
    rw [tensorμ_val, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.copy_prod)
  discard_tensor X Y := Subtype.ext (by
    change RewardKernel.lift (Kernel.discard (X.carrier × Y.carrier)) =
      ((RewardKernel.lift (Kernel.discard X.carrier)).tensor
        (RewardKernel.lift (Kernel.discard Y.carrier))).comp
        (RewardKernel.lift (Kernel.ofFun (Equiv.punitProd Unit)))
    rw [RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.discard_prod)
  copy_unit := Subtype.ext rfl
  discard_unit := Subtype.ext (by
    change RewardKernel.lift (Kernel.discard Unit) = RewardKernel.lift (Kernel.idK Unit)
    exact congrArg RewardKernel.lift Kernel.discard_unit)

/-- Composing with discard sums the reward over the output. -/
theorem comp_discard_val (f : X → Y) :
    (f ≫ ε[Y]).val = ⟨Kernel.discard X.carrier, fun x _ ⇒ ∑ y, f.val.W x y⟩ := by
  change f.val.comp (RewardKernel.discard Y.carrier) = _
  rw [RewardKernel.comp_discard]
  congr 1
  exact Kernel.Normalised.comp_discard f.2.2

/-- Discard is natural exactly for the morphisms whose total reward from every input is
zero. -/
theorem comp_discard_eq_iff (f : X → Y) : f ≫ ε[Y] = ε[X] ↔ ∀ x, ∑ y, f.val.W x y = 0 := by
  rw [← Subtype.coe_inj, comp_discard_val]
  constructor
  · intro h x
    exact congrFun (congrFun (congrArg RewardKernel.W h) x) ()
  · intro h
    ext1
    · rfl
    · funext x u
      exact h x

/-- The one-point object. -/
abbrev unitObj : FinRew := ⟨Unit⟩

/-- The morphism that stays put and earns reward one. -/
def unitReward : unitObj → unitObj :=
  ⟨RewardKernel.Examples.unitReward, RewardKernel.Examples.unitReward_stochastic⟩

/-- `FinRew` is a copy-discard category but not a Markov category: discard is not natural.
```

```
/-- The witness is the morphism `unitObj → unitObj` that stays put and earns reward one. -/
theorem discard_not_natural : unitReward ≫ ε[unitObj] ≠ ε[unitObj] := by
  rw [Ne, comp_discard_eq_iff]
  intro h
  have := h ()
  simp [unitReward, RewardKernel.Examples.unitReward] at this

/-- The embedding of `FinStoch` with zero reward. -/
def ofFinStoch (X : FinStoch) : FinRew := ⟨X.carrier⟩

/-- A stochastic kernel as a morphism with zero reward. -/
def liftHom {X Y : FinStoch} (f : X → Y) : ofFinStoch X → ofFinStoch Y := lift f.1 f.2

theorem liftHom_comp {X Y Z : FinStoch} (f : X → Y) (g : Y → Z) :
    liftHom (f ≫ g) = liftHom f ≫ liftHom g :=
  Subtype.ext (RewardKernel.lift_comp_lift f.1 g.1).symm

theorem liftHom_id (X : FinStoch) : liftHom (𝟙 X) = 𝟙 (ofFinStoch X) := rfl

theorem liftHom_tensor {X Y X' Y' : FinStoch} (f : X → Y) (g : X' → Y') :
    (liftHom (f ⊗ₘ g) : ofFinStoch (X ⊗ X') → ofFinStoch (Y ⊗ Y')) =
      (liftHom f ⊗ₘ liftHom g : ofFinStoch X ⊗ ofFinStoch X' → _) :=
  Subtype.ext (RewardKernel.lift_tensor_lift f.1 g.1).symm

theorem liftHom_injective {X Y : FinStoch} : Function.Injective (liftHom (X := X) (Y := Y)) :=
  fun _ _ h ⇒ Subtype.ext (congrArg (fun r : ofFinStoch X → ofFinStoch Y ⇒ r.1.K) h)

end FinRew

end OpenInfluence
```

Expected utility in context and the contextual identity

```
import CategoricalInfluenceDiagramsProofs.RewardKernel
```

For a reward kernel `f = (K, W) : X → Y`, a boundary prior `μ` on `X` and a terminal continuation value `c` on `Y` (both arbitrary real functions), the expected utility is

`EU_f(μ, c) = Σ_x μ x · Σ_y (W x y + K x y · c y)`,

the bundle's `Score_{μ,c}(K, W)`. A suffix `g : Y → Z` turns a continuation `c` on `Z` into the continuation `c_g y = Σ_z (W_g y z + K_g y z · c z)` on `Y`, its value function. A prefix `f` turns a prior `μ` on `X` into the induced boundary law `ν_f y = Σ_x μ x · K_f x y` and earns the prefix reward `L_f = Σ_x μ x · Σ_y W_f x y`.

The **contextual identity** (ADR 0018; the bundle's `INF-FIXED-CONTEXT`) is

`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`.

The second equation holds for arbitrary tables (`eu_continuation`). The first needs the suffix's probability component to be normalised (`eu_comp`): the prefix reward `W_f x y` must survive the suffix with total mass one. `eu_comp` shows exactly that the defect is `Σ_x μ x Σ_y W_f x y (Σ_z K_g y z - 1)` (`eu_comp_general`), so the hypothesis cannot simply be dropped. For an interpreted diagram, where `f` and `g` are the reward kernels of the two components under fixed strategies, `Interpretation.lean` applies this through the functoriality of `interpret`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite

namespace RewardKernel

variable {X Y Z : Type} [Fintype X] [Fintype Y] [Fintype Z]
```

```
/-- Expected utility of a reward kernel under the boundary prior `μ` and the terminal
continuation `c`. -/
def eu (f : RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) : ℝ :=
  ∑ x, μ x * ∑ y, (f.W x y + f.K x y * c y)

/-- The value function `c_g` of a suffix `g` with terminal continuation `c`. -/
def continuation (g : RewardKernel Y Z) (c : Z → ℝ) : Y → ℝ :=
  fun y ⇒ ∑ z, (g.W y z + g.K y z * c z)

/-- The boundary law `ν_f = μ K_f` induced by the prefix `f` from the prior `μ`. -/
def pushforward (f : RewardKernel X Y) (μ : X → ℝ) : Y → ℝ :=
  fun y ⇒ ∑ x, μ x * f.K x y

/-- The reward `L_f = Σ μ W_f` earned by the prefix `f` from the prior `μ`. -/
def prefixReward (f : RewardKernel X Y) (μ : X → ℝ) : ℝ :=
  ∑ x, μ x * ∑ y, f.W x y

/-- The expected utility with terminal continuation `c` is the prior-weighted continuation
of `f`. -/
theorem eu_eq_continuation (f : RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) :
    f.eu μ c = ∑ x, μ x * f.continuation c x := rfl

/-- Composition without normalisation: the composite's expected utility differs from the
contextual one by the prefix reward times the suffix's missing mass. -/
theorem eu_comp_general (f : RewardKernel X Y) (g : RewardKernel Y Z) (μ : X → ℝ)
    (c : Z → ℝ) :
    (f.comp g).eu μ c =
      f.eu μ (g.continuation c) + ∑ x, μ x * ∑ y, f.W x y * ((∑ z, g.K y z) - 1) := by
  have hL : ∀ x, (∑ z, ((f.comp g).W x z + (f.comp g).K x z * c z)) =
      ∑ y, ∑ z, (f.W x y * g.K y z + f.K x y * g.W y z + f.K x y * g.K y z * c z) := by
    intro x
    simp only [comp_W, comp_K, Kernel.comp, Pi.add_apply, Finset.sum_mul,
      ← Finset.sum_add_distrib]
    exact Finset.sum_comm
  have hR : ∀ x y, (f.W x y + f.K x y * g.continuation c y) +
      f.W x y * ((∑ z, g.K y z) - 1) =
      ∑ z, (f.W x y * g.K y z + f.K x y * g.W y z + f.K x y * g.K y z * c z) := by
    intro x y
    have e : (∑ z, (f.W x y * g.K y z + f.K x y * g.W y z + f.K x y * g.K y z * c z)) =
        f.W x y * ∑ z, g.K y z + f.K x y * g.continuation c y := by
      simp only [continuation, Finset.mul_sum, ← Finset.sum_add_distrib, mul_add, mul_assoc,
        add_assoc]
    rw [e]
    ring
  simp only [eu, ← Finset.sum_add_distrib, ← mul_add, hL, ← hR]

/-- First half of the contextual identity: with a normalised suffix, the composite's
expected utility is the prefix's expected utility against the suffix's value function. -/
theorem eu_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (hg : Kernel.Normalised g.K)
    (μ : X → ℝ) (c : Z → ℝ) : (f.comp g).eu μ c = f.eu μ (g.continuation c) := by
  rw [eu_comp_general]
  have h0 : ∀ y, (∑ z, g.K y z) - 1 = 0 := fun y ⇒ by rw [hg y, sub_self]
  simp [h0]

/-- Second half of the contextual identity: against any continuation `c'`, the prefix's
expected utility is its own reward plus the continuation's expectation under the induced
boundary law. -/
theorem eu_continuation (f : RewardKernel X Y) (g : RewardKernel Y Z) (μ : X → ℝ)
    (c : Z → ℝ) :
    f.eu μ (g.continuation c) = f.prefixReward μ + g.eu (f.pushforward μ) c := by
  rw [eu_eq_continuation g]
  generalize g.continuation c = C
  simp only [eu, prefixReward, pushforward, mul_add, Finset.sum_add_distrib, Finset.mul_sum,
    Finset.sum_mul]
  congr 1
```

```
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun y _ ⇒ Finset.sum_congr rfl fun x _ ⇒ by ring

/-- **Contextual identity** for reward kernels:
`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`, for a suffix with normalised `K`. -/
theorem contextual_identity (f : RewardKernel X Y) (g : RewardKernel Y Z)
    (hg : Kernel.Normalised g.K) (μ : X → ℝ) (c : Z → ℝ) :
    (f.comp g).eu μ c = f.eu μ (g.continuation c) ∧
      f.eu μ (g.continuation c) = f.prefixReward μ + g.eu (f.pushforward μ) c :=
  ⟨eu_comp f g hg μ c, eu_continuation f g μ c⟩

/-- The normalisation of the suffix cannot be dropped from the first half: here the suffix
loses all mass, and the prefix's reward of one is lost with it. -/
theorem eu_comp_needs_normalised :
    ∃ (f g : RewardKernel Unit Unit) (μ : Unit → ℝ) (c : Unit → ℝ),
      (f.comp g).eu μ c ≠ f.eu μ (g.continuation c) := by
  refine ⟨⟨fun _ _ ⇒ 1, fun _ _ ⇒ 1⟩, ⟨fun _ _ ⇒ 0, fun _ _ ⇒ 0⟩, fun _ ⇒ 1,
    fun _ ⇒ 0, ?_⟩
  simp [eu, comp, continuation, Kernel.comp]

end RewardKernel

end OpenInfluence
```

Decision and utility generators on the typed open-network syntax

```
import CategoricalInfluenceDiagramsProofs.Basic
```

ADR 0010's typed open-network syntax (`OpenNet.Net X Y`) has one kind of generator: a mechanism with a label, an ordered list of typed parents and one typed output. This module extends it in the two ways ADR 0018 (decision 5) asks for.

- **Decision generators.** An influence-diagram signature (`IDSignature`) marks each generator label as a chance mechanism or a decision (`isDecision`). A decision generator is an ordinary generator of the underlying net whose label is a decision label; its ordered parent list is its list of **information ports**, so a policy for it is a function of the values on those ports (`Interpretation.lean`).
- **Utility generators.** A signature also has utility labels, each with an ordered list of input types (`uinputs`). An open influence diagram (`IDNet X Y`) is a net together with a finite set of utility generators, each with a label and an ordered, typed list of the net's variables (its **scope**). A utility reads its scope and produces no variable, so utilities never cross a boundary; their reward crosses only through the semantics, as ADR 0018 requires.

Serial composition (`IDNet.comp`) and tensor (`IDNet.tensor`) are those of the underlying nets, with the utility generators of the two components placed side by side and their scopes substituted along the nets' variable maps (`Net.left`, `Net.right`, `Net.tinl`, `Net.tinr`). Identities have no generators and no utilities. Every net embeds with no utility (`IDNet.ofNet`), and the atomic generators are `IDNet.generator` (one chance or decision generator on its own inputs) and `IDNet.utility` (one utility on a scope of an interface, passing the interface through).

Scope. The syntax is the full generality of `OpenNet.Net`: arbitrary finite nets, not only terms built from generators. It is **not** quotiented by structural isomorphism, so the functoriality theorems of `Interpretation.lean` are equations on representatives rather than a Mathlib `Functor` out of a syntax category. Observability flags (`Basic.lean`) are not part of this syntax: the fixed-strategy semantics does not read them, and information locality is a separate side condition on strategies.

```
set_option autoImplicit false

namespace OpenInfluence

/-- A signature for open influence diagrams: an `OpenNet` signature whose labels are
chance mechanisms or decisions, together with utility labels and their input types. -/
structure IDSignature extends OpenNet.Signature where
  /-- Whether generators with this label are decisions (`true`) or chance mechanisms. -/
  isDecision : Label → Bool
```

```
  /-- Utility labels. -/
  ULabel : Type
  /-- The ordered input types of a utility label. -/
  uinputs : ULabel → List Ty

/-- The finite typed interfaces of a signature, those of the underlying `OpenNet`
signature. -/
abbrev IDSignature.Iface (S : IDSignature) := OpenNet.Interface S.toSignature

variable {S : IDSignature}

/-- An open influence diagram `X → Y`: a typed open net (whose generators are chance
mechanisms and decisions) with a finite set of utility generators, each reading an ordered,
typed list of the net's variables. -/
structure IDNet (X Y : S.Iface) where
  /-- The underlying open net of chance and decision generators. -/
  net : OpenNet.Net X Y
  /-- The utility generators. -/
  UGen : Type
  /-- There are finitely many utility generators. -/
  ufinite : Fintype UGen
  /-- The label of each utility generator. -/
  ulabel : UGen → S.ULabel
  /-- The ordered scope of each utility generator: input ports or internal generators. -/
  uparents : UGen → List net.Var
  /-- Each scope has the input types of its label. -/
  uparents_typed : ∀ u, (uparents u).map net.varType = S.uinputs (ulabel u)

attribute [instance] IDNet.ufinite

namespace IDNet

variable {X Y Z X' Y' : S.Iface}

/-- A generator of the underlying net is a decision when its label is a decision label. -/
def IsDecision (f : IDNet X Y) (a : f.net.Gen) : Prop := S.isDecision (f.net.label a) = true

/-- The information ports of a generator: its ordered parents. -/
def info (f : IDNet X Y) (a : f.net.Gen) : List f.net.Var := f.net.parents a

/-- An open net with no utility generators. -/
def ofNet (n : OpenNet.Net X Y) : IDNet X Y where
  net := n
  UGen := Empty
  ufinite := inferInstance
  ulabel := Empty.elim
  uparents := Empty.elim
  uparents_typed u := u.elim

/-- The identity: pure wiring, with no generators and no utilities. -/
def identity (X : S.Iface) : IDNet X X := ofNet (OpenNet.Net.identity X)

/-- Serial composition: compose the nets, and keep the utilities of both components with
their scopes substituted along the gluing. -/
def comp (f : IDNet X Y) (g : IDNet Y Z) : IDNet X Z where
  net := f.net.comp g.net
  UGen := f.UGen ⊕ g.UGen
  ufinite := inferInstance
  ulabel := Sum.elim f.ulabel g.ulabel
  uparents := Sum.elim
    (fun u ⇒ (f.uparents u).map (OpenNet.Net.left f.net g.net))
    (fun u ⇒ (g.uparents u).map (OpenNet.Net.right f.net g.net))
  uparents_typed := by
    intro u
    cases u with
    | inl u ⇒
```

```
      change ((f.uparents u).map (OpenNet.Net.left f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (f.uparents u).map k)
        (funext (OpenNet.Net.left_typed f.net g.net))).trans (f.uparents_typed u)
    | inr u ⇒
      change ((g.uparents u).map (OpenNet.Net.right f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (g.uparents u).map k)
        (funext (OpenNet.Net.right_typed f.net g.net))).trans (g.uparents_typed u)

/-- Tensor: place the nets side by side, and keep the utilities of both components with
their scopes reindexed into the disjoint union. -/
def tensor (f : IDNet X Y) (g : IDNet X' Y') : IDNet (X.sum X') (Y.sum Y') where
  net := f.net.tensor g.net
  UGen := f.UGen ⊕ g.UGen
  ufinite := inferInstance
  ulabel := Sum.elim f.ulabel g.ulabel
  uparents := Sum.elim
    (fun u ⇒ (f.uparents u).map (OpenNet.Net.tinl f.net g.net))
    (fun u ⇒ (g.uparents u).map (OpenNet.Net.tinr f.net g.net))
  uparents_typed := by
    intro u
    cases u with
    | inl u ⇒
      change ((f.uparents u).map (OpenNet.Net.tinl f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (f.uparents u).map k)
        (funext (OpenNet.Net.tinl_typed f.net g.net))).trans (f.uparents_typed u)
    | inr u ⇒
      change ((g.uparents u).map (OpenNet.Net.tinr f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (g.uparents u).map k)
        (funext (OpenNet.Net.tinr_typed f.net g.net))).trans (g.uparents_typed u)

@[simp] theorem comp_net (f : IDNet X Y) (g : IDNet Y Z) : (f.comp g).net = f.net.comp g.net :=
  rfl

@[simp] theorem tensor_net (f : IDNet X Y) (g : IDNet X' Y') :
    (f.tensor g).net = f.net.tensor g.net := rfl
```

Atomic generators

```
/-- The interface of the ordered input types of a generator label. -/
def inputIface (l : S.Label) : S.Iface :=
  ⟨Fin (S.inputs l).length, inferInstance, fun i ⇒ (S.inputs l).get i⟩

/-- The one-port interface of the output type of a generator label. -/
def outputIface (l : S.Label) : S.Iface := ⟨Unit, inferInstance, fun _ ⇒ S.result l⟩

/-- A single chance or decision generator labelled `l`, reading its input ports in order
(for a decision, its information ports) and exposing its output. -/
def generator (l : S.Label) : IDNet (inputIface l) (outputIface l) :=
  ofNet {
    Gen := Unit
    finite := inferInstance
    label := fun _ ⇒ l
    parents := fun _ ⇒ List.ofFn (fun i ⇒ Sum.inl i)
    output := fun _ ⇒ Sum.inr ()
    parents_typed := fun _ ⇒ by
      rw [List.map_ofFn]
      exact List.ofFn_get (S.inputs l)
    output_typed := fun _ ⇒ rfl
    acyclic := ⟨fun _ ⇒ 0, fun _ _ h ⇒ by simp at h⟩ }

/-- A single utility generator labelled `u` reading the ports `scope` of `X`, which it
```

```
passes through unchanged. -/
def utility (X : S.Iface) (u : S.ULabel) (scope : List X.Port)
    (h : scope.map X.type = S.uinputs u) : IDNet X X where
  net := OpenNet.Net.identity X
  UGen := Unit
  ufinite := inferInstance
  ulabel _ := u
  uparents _ := scope.map Sum.inl
  uparents_typed _ := by
    rw [List.map_map]
    exact h

end IDNet

end OpenInfluence
```

The fixed-strategy interpretation and its functoriality

```
import CategoricalInfluenceDiagramsProofs.Syntax
import CategoricalInfluenceDiagramsProofs.Context
import CategoricalInfluenceDiagramsProofs.Category
```

An **interpretation** of an influence-diagram signature (`IDInterpretation`) gives every type a finite state space, every chance label a nonnegative table normalised on every correctly typed parent row, and every utility label a real-valued table on its ordered inputs. A **strategy** (`IDInterpretation.Strategy`) gives every decision label a deterministic policy: a function from the values on the decision's information ports to a state of its output type. As in `CategoricalInfluenceDiagrams.jl`, policies are keyed by label, so decision generators with equal labels share a policy, and one strategy covers both components of a composite (the Julia package's disjoint union of the components' strategies, when their decision labels are disjoint).

A strategy turns the interpretation into an ordinary `OpenNet.Interpretation` (`IDInterpretation.interp`): a decision generator becomes the point mass at its policy's choice, a chance generator keeps its table. Its sum-product kernel (ADR 0010, unchanged) is the probability component `K` of the interpretation. The reward component `W` sums, over the same internal assignments, their probability times the total utility of the diagram's utility generators read on their scopes (`rewardTable`). So the fixed-strategy semantics of an open influence diagram `f : X → Y` is the reward kernel

`interpret σ f = (K_f, W_f) : RewardKernel (Boundary X) (Boundary Y)`,

ADR 0018's `interpret(o, strategy)`. Its probability component is stochastic (`interpret_stochastic`), so it is a morphism of `FinRew` (`interpretHom`).

Functoriality for a fixed strategy, on representatives (the syntax is not quotiented):

- `interpret_comp`: `interpret σ (f ; g) = interpret σ f ; interpret σ g`;
- `interpret_identity`: `interpret σ (id X) = (id, 0)`;
- `interpret_tensor`: `interpret σ (f ⊗ g)`, read through the canonical bijection between assignments of `X ⊔ X'` and pairs of assignments, is `interpret σ f ⊗ interpret σ g`; `interpret_tensor_natural` states the same as naturality of that bijection, the shape of `OpenNet.Interpretation.tensor_natural`.

The **contextual identity** then holds for interpreted diagrams (`contextual_identity`), with the normalisation of the suffix supplied by `interpret_stochastic`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open CategoryTheory FiniteKernelsProofs.Finite OpenNet

/-- A semantics for the chance and utility labels of an influence-diagram signature. -/
structure IDInterpretation (S : IDSignature) where
  /-- The finite state space of each type. -/
```

```
  states : S.Ty → Type
  /-- State spaces are finite. -/
  finite : ∀ t, Fintype (states t)
  /-- The table of a chance label, on ordered typed parent values and one child value. -/
  chance : S.Label → List (Sigma states) → Sigma states → ℝ
  /-- Chance tables are nonnegative. -/
  chance_nonneg : ∀ l p s, S.isDecision l = false → 0 ≤ chance l p s
  /-- Chance tables are normalised on every correctly typed parent row. -/
  chance_normalized : ∀ l p, S.isDecision l = false → p.map Sigma.fst = S.inputs l →
    @Finset.sum (states (S.result l)) ℝ _ (@Finset.univ _ (finite _))
      (fun s ⇒ chance l p ⟨S.result l, s⟩) = 1
  /-- The table of a utility label, on ordered typed input values. -/
  utility : S.ULabel → List (Sigma states) → ℝ

attribute [instance] IDInterpretation.finite

namespace IDInterpretation

variable {S : IDSignature} (M : IDInterpretation S)

/-- A deterministic strategy: for each decision label, a policy mapping the values on the
decision's information ports to a state of its output type. -/
def Strategy : Type :=
  (l : S.Label) → S.isDecision l = true → List (Sigma M.states) → M.states (S.result l)

open Classical in
/-- The `OpenNet` interpretation for a fixed strategy: decisions are the point masses of
their policies, chance mechanisms keep their tables. -/
def interp (σ : M.Strategy) : OpenNet.Interpretation S.toSignature where
  states := M.states
  finite := M.finite
  weight l p s :=
    if h : S.isDecision l = true then (if s = ⟨S.result l, σ l h p⟩ then 1 else 0)
    else M.chance l p s
  nonneg l p s := by
    by_cases h : S.isDecision l = true
    · simp only [h, dite_true]
      split_ifs <;> norm_num
    · simp only [h]
      exact M.chance_nonneg l p s (by simpa using h)
  normalized l p hp := by
    by_cases h : S.isDecision l = true
    · simp only [h, dite_true, Sigma.mk.injEq, heq_eq_eq, true_and]
      simp
    · simp only [h]
      exact M.chance_normalized l p (by simpa using h) hp

variable (σ : M.Strategy)
variable {X Y Z X' Y' : S.Iface}

/-- The total utility of the diagram's utility generators, read on their scopes, at the
input assignment `x` and the internal assignment `z`. -/
def reward (f : IDNet X Y) (x : (M.interp σ).Boundary X) (z : (M.interp σ).Internal f.net) :
    ℝ :=
  ∑ u, M.utility (f.ulabel u) ((f.uparents u).map ((M.interp σ).valuation f.net x z).value)

/-- The reward table: the probability-weighted total utility of the internal assignments
that produce each output. -/
def rewardTable (f : IDNet X Y) :
    Kernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  fun x y ⇒ ∑ z, if y = (M.interp σ).read f.net x z then
    (M.interp σ).joint f.net x z * M.reward σ f x z else 0

/-- The fixed-strategy semantics of an open influence diagram: the sum-product kernel of
its net and its reward table. -/
def interpret (f : IDNet X Y) :
```

```
    RewardKernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  ⟨(M.interp σ).kernel f.net, M.rewardTable σ f⟩

theorem interpret_stochastic (f : IDNet X Y) : (M.interpret σ f).Stochastic :=
  (M.interp σ).kernel_stochastic f.net

/-- The boundary assignments of an interface, as an object of `FinRew`. -/
def space (X : S.Iface) : FinRew := ⟨(M.interp σ).Boundary X⟩

/-- The interpretation as a morphism of `FinRew`. -/
def interpretHom (f : IDNet X Y) : M.space σ X → M.space σ Y :=
  ⟨M.interpret σ f, M.interpret_stochastic σ f⟩
```

The atomic generators

```
open Classical in
/-- A decision generator is the point mass at its policy's choice, read on the values of
its information ports. -/
theorem interp_weight_decision (l : S.Label) (h : S.isDecision l = true)
    (p : List (Sigma M.states)) (s : Sigma M.states) :
    (M.interp σ).weight l p s = if s = ⟨S.result l, σ l h p⟩ then 1 else 0 := by
  simp [interp, h]

/-- A chance generator keeps its table. -/
theorem interp_weight_chance (l : S.Label) (h : S.isDecision l = false)
    (p : List (Sigma M.states)) (s : Sigma M.states) :
    (M.interp σ).weight l p s = M.chance l p s := by
  simp [interp, h]

/-- A utility generator passes its interface through unchanged. -/
theorem interpret_utility_K (X : S.Iface) (u : S.ULabel) (scope : List X.Port)
    (h : scope.map X.type = S.uinputs u) :
    (M.interpret σ (IDNet.utility X u scope h)).K = Kernel.idK _ := by
  change (M.interp σ).kernel (Net.wire id (fun _ ⇒ rfl)) = Kernel.idK _
  rw [(M.interp σ).kernel_wire]
  apply Kernel.ofFun_congr
  intro x
  rfl

/-- A utility generator earns, on the diagonal, the utility of its label read on its
scope. -/
theorem interpret_utility_W (X : S.Iface) (u : S.ULabel) (scope : List X.Port)
    (h : scope.map X.type = S.uinputs u) (x y : (M.interp σ).Boundary X) :
    (M.interpret σ (IDNet.utility X u scope h)).W x y =
      if y = x then M.utility u (scope.map x.value) else 0 := by
  letI : Unique ((M.interp σ).Internal (Net.wire (id : X.Port → X.Port) (fun _ ⇒ rfl))) := by
    change Unique ((M.interp σ).Assignment (S.result ∘ (Empty.elim : Empty → S.Label)))
    infer_instance
  change (∑ z, if y = (M.interp σ).read (Net.wire id (fun _ ⇒ rfl)) x z then
      (M.interp σ).joint (Net.wire id (fun _ ⇒ rfl)) x z *
        M.reward σ (IDNet.utility X u scope h) x z else 0) = _
  rw [Fintype.sum_unique, (M.interp σ).joint_wire, one_mul]
  have hread : (M.interp σ).read (Net.wire id (fun _ ⇒ rfl)) x default = x := rfl
  have hrew : M.reward σ (IDNet.utility X u scope h) x
      (default : (M.interp σ).Internal (Net.wire (id : X.Port → X.Port) (fun _ ⇒ rfl))) =
      M.utility u (scope.map x.value) := by
    unfold reward
    change (∑ _v : Unit, M.utility u ((scope.map Sum.inl).map _)) = _
    rw [Fintype.sum_unique, List.map_map]
    rfl
  rw [hread, hrew]
```

Rewards of composites

```
theorem reward_comp (f : IDNet X Y) (g : IDNet Y Z) (x : (M.interp σ).Boundary X)
    (a : (M.interp σ).Internal f.net) (b : (M.interp σ).Internal g.net) :
    M.reward σ (f.comp g) x ((M.interp σ).combine f.net g.net a b) =
      M.reward σ f x a + M.reward σ g ((M.interp σ).read f.net x a) b := by
  unfold reward
  erw [Fintype.sum_sum_type]
  congr 1
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (f.ulabel u)
      (((f.uparents u).map (Net.left f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (f.uparents u).map k)
      (funext ((M.interp σ).valuation_comp_left f.net g.net x a b))
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (g.ulabel u)
      (((g.uparents u).map (Net.right f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (g.uparents u).map k)
      (funext ((M.interp σ).valuation_comp_right f.net g.net x a b))

theorem reward_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (a : (M.interp σ).Internal f.net)
    (b : (M.interp σ).Internal g.net) :
    M.reward σ (f.tensor g) (Interpretation.Assignment.join _ x x')
        ((M.interp σ).combine f.net g.net a b) =
      M.reward σ f x a + M.reward σ g x' b := by
  unfold reward
  erw [Fintype.sum_sum_type]
  congr 1
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (f.ulabel u)
      (((f.uparents u).map (Net.tinl f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (f.uparents u).map k)
      (funext ((M.interp σ).valuation_tensor_left f.net g.net x x' a b))
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (g.ulabel u)
      (((g.uparents u).map (Net.tinr f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (g.uparents u).map k)
      (funext ((M.interp σ).valuation_tensor_right f.net g.net x x' a b))
```

Functoriality under serial composition

```
/-- Summing a product of two output-fibre sums over the middle boundary substitutes the
first component's output into the second. -/
theorem sum_fibre_mul {f : Net X Y} {g : Net Y Z} (x : (M.interp σ).Boundary X)
    (w : (M.interp σ).Boundary Z) (F : (M.interp σ).Internal f → ℝ)
    (G : (M.interp σ).Boundary Y → (M.interp σ).Internal g → ℝ) :
    (∑ y, (∑ a, if y = (M.interp σ).read f x a then F a else 0) *
        (∑ b, if w = (M.interp σ).read g y b then G y b else 0)) =
      ∑ a, ∑ b, if w = (M.interp σ).read g ((M.interp σ).read f x a) b then
        F a * G ((M.interp σ).read f x a) b else 0 := by
  simp only [Finset.sum_mul]
  rw [Finset.sum_comm]
```

```
  apply Finset.sum_congr rfl
  intro a _
  rw [Finset.sum_eq_single ((M.interp σ).read f x a)]
  · simp only [if_true, Finset.mul_sum, mul_ite, mul_zero]
  · intro y _ hy
    simp [hy]
  · simp

theorem kernel_eq_sum (f : Net X Y) (x : (M.interp σ).Boundary X)
    (y : (M.interp σ).Boundary Y) :
    (M.interp σ).kernel f x y =
      ∑ z, if y = (M.interp σ).read f x z then (M.interp σ).joint f x z else 0 := by
  unfold Interpretation.kernel
  rfl

theorem rewardTable_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.rewardTable σ (f.comp g) =
      Kernel.comp (M.rewardTable σ f) ((M.interp σ).kernel g.net) +
        Kernel.comp ((M.interp σ).kernel f.net) (M.rewardTable σ g) := by
  funext x w
  simp only [Pi.add_apply, Kernel.comp, kernel_eq_sum]
  unfold rewardTable
  rw [M.sum_fibre_mul σ x w (fun a ⇒ (M.interp σ).joint f.net x a * M.reward σ f x a)
      (fun y b ⇒ (M.interp σ).joint g.net y b),
    M.sum_fibre_mul σ x w (fun a ⇒ (M.interp σ).joint f.net x a)
      (fun y b ⇒ (M.interp σ).joint g.net y b * M.reward σ g y b)]
  rw [← Finset.sum_add_distrib]
  change (∑ z : (M.interp σ).Internal (f.net.comp g.net),
      if w = (M.interp σ).read (f.net.comp g.net) x z then
        (M.interp σ).joint (f.net.comp g.net) x z * M.reward σ (f.comp g) x z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_comp f.net g.net x a b
  have hj := (M.interp σ).joint_comp f.net g.net x a b
  have hw := M.reward_comp σ f g x a b
  simp only [Interpretation.combine] at hr hj hw
  rw [hr, hj, hw]
  split_ifs <;> ring

/-- **Functoriality under serial composition**: the interpretation of a composite is the
composite, in the expectation-semiring kernel category, of the interpretations. -/
theorem interpret_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.interpret σ (f.comp g) = (M.interpret σ f).comp (M.interpret σ g) := by
  ext1
  · exact (M.interp σ).kernel_comp f.net g.net
  · exact M.rewardTable_comp σ f g

theorem interpretHom_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.interpretHom σ (f.comp g) = M.interpretHom σ f ≫ M.interpretHom σ g :=
  Subtype.ext (M.interpret_comp σ f g)

/-- **Identities**: pure wiring with no utilities is interpreted as `(id, 0)`. -/
theorem interpret_identity (X : S.Iface) :
    M.interpret σ (IDNet.identity X) = RewardKernel.id ((M.interp σ).Boundary X) := by
  ext1
  · change (M.interp σ).kernel (Net.wire id (fun _ ⇒ rfl)) = Kernel.idK _
    rw [(M.interp σ).kernel_wire]
    apply Kernel.ofFun_congr
    intro x
    rfl
  · funext x y
```

```
    have hr : ∀ z, M.reward σ (IDNet.identity X) x z = 0 :=
      fun z ⇒ Finset.sum_eq_zero (fun u _ ⇒ (u : Empty).elim)
    change M.rewardTable σ (IDNet.identity X) x y = 0
    unfold rewardTable
    simp [hr]

theorem interpretHom_identity (X : S.Iface) :
    M.interpretHom σ (IDNet.identity X) = 𝟙 (M.space σ X) :=
  Subtype.ext (M.interpret_identity σ X)
```

Functoriality under tensor

```
/-- The canonical bijection between pairs of boundary assignments and assignments of the
disjoint-union interface. -/
def join (X X' : S.Iface) :
    (M.interp σ).Boundary X × (M.interp σ).Boundary X' ≃ (M.interp σ).Boundary (X.sum X') :=
  (Interpretation.Assignment.sumEquiv (M.interp σ) X.type X'.type).symm

@[simp] theorem join_apply (X X' : S.Iface) (p : (M.interp σ).Boundary X ×
    (M.interp σ).Boundary X') :
    M.join σ X X' p = Interpretation.Assignment.join _ p.1 p.2 := rfl

theorem rewardTable_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (y : (M.interp σ).Boundary Y)
    (y' : (M.interp σ).Boundary Y') :
    M.rewardTable σ (f.tensor g) (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      M.rewardTable σ f x y * (M.interp σ).kernel g.net x' y' +
        (M.interp σ).kernel f.net x y * M.rewardTable σ g x' y' := by
  simp only [kernel_eq_sum]
  unfold rewardTable
  rw [Finset.sum_mul_sum, Finset.sum_mul_sum, ← Finset.sum_add_distrib]
  change (∑ z : (M.interp σ).Internal (f.net.tensor g.net),
      if Interpretation.Assignment.join _ y y' =
          (M.interp σ).read (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z then
        (M.interp σ).joint (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z *
          M.reward σ (f.tensor g) (Interpretation.Assignment.join _ x x') z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_tensor f.net g.net x x' a b
  have hj := (M.interp σ).joint_tensor f.net g.net x x' a b
  have hw := M.reward_tensor σ f g x x' a b
  simp only [Interpretation.combine] at hr hj hw
  rw [hr, hj, hw]
  by_cases h₁ : y = (M.interp σ).read f.net x a <;>
    by_cases h₂ : y' = (M.interp σ).read g.net x' b <;>
      simp [Interpretation.Assignment.join_injective, h₁, h₂]
  ring

theorem kernel_tensor_join (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (y : (M.interp σ).Boundary Y)
    (y' : (M.interp σ).Boundary Y') :
    (M.interp σ).kernel (f.tensor g).net (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      (M.interp σ).kernel f.net x y * (M.interp σ).kernel g.net x' y' :=
  (M.interp σ).kernel_tensor f.net g.net x x' y y'

/-- Read a reward kernel through maps on its input and output states. -/
def _root_.OpenInfluence.RewardKernel.reindex {A B C D : Type} (r : RewardKernel C D)
    (i : A → C) (o : B → D) : RewardKernel A B :=
```

```
  ⟨fun a b ⇒ r.K (i a) (o b), fun a b ⇒ r.W (i a) (o b)⟩

/-- **Functoriality under tensor**: read through the canonical bijections between
assignments of a disjoint-union interface and pairs of assignments, the interpretation of
a tensor is the tensor of the interpretations. -/
theorem interpret_tensor (f : IDNet X Y) (g : IDNet X' Y') :
    (M.interpret σ (f.tensor g)).reindex (M.join σ X X') (M.join σ Y Y') =
      (M.interpret σ f).tensor (M.interpret σ g) := by
  ext1
  · funext p q
    exact M.kernel_tensor_join σ f g p.1 p.2 q.1 q.2
  · funext p q
    exact M.rewardTable_tensor σ f g p.1 p.2 q.1 q.2

/-- **Functoriality under tensor**, as naturality of the canonical bijection (the shape of
`OpenNet.Interpretation.tensor_natural`). -/
theorem interpret_tensor_natural (f : IDNet X Y) (g : IDNet X' Y') :
    ((M.interpret σ f).tensor (M.interpret σ g)).comp (RewardKernel.ofFun (M.join σ Y Y')) =
      (RewardKernel.ofFun (M.join σ X X')).comp (M.interpret σ (f.tensor g)) := by
  rw [← M.interpret_tensor σ f g]
  ext1
  · simp only [RewardKernel.comp_K, RewardKernel.ofFun, RewardKernel.lift_K,
      Kernel.comp_ofEquiv_right, Kernel.comp_ofFun_left, RewardKernel.reindex]
    funext p q
    simp
  · simp only [RewardKernel.comp_W, RewardKernel.ofFun, RewardKernel.lift_K,
      RewardKernel.lift_W, KernelAlgebra.kcomp_zero_left, KernelAlgebra.kcomp_zero_right,
      add_zero, zero_add, Kernel.comp_ofEquiv_right, Kernel.comp_ofFun_left,
      RewardKernel.reindex]
    funext p q
    simp
```

The contextual identity for interpreted diagrams

```
/-- **Contextual identity** for open influence diagrams under a fixed strategy: for every
boundary prior `μ` and terminal continuation `c`,
`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`, where `c_g` is the suffix's value
function, `ν_f` the boundary law induced by the prefix and `L_f` the prefix's reward. -/
theorem contextual_identity (f : IDNet X Y) (g : IDNet Y Z)
    (μ : (M.interp σ).Boundary X → ℝ) (c : (M.interp σ).Boundary Z → ℝ) :
    (M.interpret σ (f.comp g)).eu μ c =
        (M.interpret σ f).eu μ ((M.interpret σ g).continuation c) ∧
      (M.interpret σ f).eu μ ((M.interpret σ g).continuation c) =
        (M.interpret σ f).prefixReward μ +
          (M.interpret σ g).eu ((M.interpret σ f).pushforward μ) c := by
  rw [M.interpret_comp σ f g]
  exact RewardKernel.contextual_identity _ _ (M.interpret_stochastic σ g).2 μ c

end IDInterpretation

end OpenInfluence
```

Optimisation does not compose component by component

```
import CategoricalInfluenceDiagramsProofs.Context
import Mathlib.Tactic.NormNum
```

The contextual identity says each component's value depends on its context: the prefix sees the suffix only through the value function `c_g`, the suffix sees the prefix only through the induced boundary law `ν_f`. This module proves that the context cannot be dropped, with the finite witness of the sealed bundle's Lean track (`INF-ISOLATED-OBSTRUCTION`, `INF-HIDDEN-OBSTRUCTION`), restated in the reward-kernel semantics and not imported from the bundle.

The witness. A fair hidden Boolean `H` and an early action `A` that observes nothing form the prefix `early a : Unit → H × A`, which chooses `A = a` and pays utility `-[A]`. The suffix `late τ : H × A → Unit` is an action `B = τ A` that observes only `A` (so `H` is hidden at the boundary) and pays utilities `4[A]` and `2[B = H]`. The closed composite from the unit prior with zero continuation has expected utility `1 + 3[a]`.

Isolated optimisation (`isolated_optimisation_fails`). Optimised on its own (unit prior, zero continuation), the prefix chooses `a = false`, since its only utility is `-[A]`. Optimised on its own against the boundary law that this prefix actually induces, every suffix policy has value `1`. So **every** pair of separately optimal components has composite value `1`, while the composite with `a = true` attains `4`, the joint optimum: `1 < 4`.

Pointwise maximisation at a hidden boundary state (`hidden_input_obstruction`). Against the same boundary law, every legal suffix policy (a function of `A` only) has value `1`. Choosing `B` to maximise reward separately at each boundary state `(H, A)` gives the selector `B = H`, whose value is `2`; it reads the hidden `H` and is not a legal policy.

Both are statements about the fixed-strategy semantics; they are not instances of the typed syntax of `Syntax.lean`, whose interpretation they would be (the theorems there reduce such a diagram to these reward kernels by `interpret_comp`).

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

namespace Obstruction

open FiniteKernelsProofs.Finite RewardKernel

/-- The boundary between the components: the hidden `H` and the observed `A`. -/
abbrev Boundary : Type := Bool × Bool

/-- The prefix with early action `A = a`: a fair hidden `H`, and utility `-[A]`. -/
def early (a : Bool) : RewardKernel Unit Boundary :=
  ⟨fun _ s ⇒ if s.2 = a then 1 / 2 else 0,
    fun _ s ⇒ if s.2 = a then 1 / 2 * (if s.2 then -1 else 0) else 0⟩

/-- The suffix whose action `B` is chosen by a selector on the whole boundary state, with
utilities `4[A]` and `2[B = H]`. -/
def lateSel (sel : Boundary → Bool) : RewardKernel Boundary Unit :=
  ⟨fun _ _ ⇒ 1, fun s _ ⇒ (if s.2 then 4 else 0) + (if sel s = s.1 then 2 else 0)⟩

/-- A legal suffix policy reads only the observed `A`. -/
def late (τ : Bool → Bool) : RewardKernel Boundary Unit := lateSel fun s ⇒ τ s.2

/-- A selector is a legal policy when it reads only the observed coordinate. -/
def Legal (sel : Boundary → Bool) : Prop := ∃ τ : Bool → Bool, ∀ s, sel s = τ s.2

/-- The unit prior on the closed input. -/
def unitPrior : Unit → ℝ := fun _ ⇒ 1

/-- The zero terminal continuation. -/
def zeroCont {Y : Type} : Y → ℝ := fun _ ⇒ 0

/-- The boundary law induced by the prefix `early a`. -/
def boundaryLaw (a : Bool) : Boundary → ℝ := (early a).pushforward unitPrior

/-- `p` is optimal among the members of a family of reward kernels for the prior `μ` and
the continuation `c`, the component considered on its own. -/
def IsolatedOptimal {X Y P : Type} [Fintype X] [Fintype Y] (family : P → RewardKernel X Y)
    (μ : X → ℝ) (c : Y → ℝ) (p : P) : Prop :=
  ∀ q, (family q).eu μ c ≤ (family p).eu μ c
```

Values

```
theorem early_eu (a : Bool) : (early a).eu unitPrior zeroCont = if a then -1 else 0 := by
  cases a <;>
    simp [eu, early, unitPrior, zeroCont, Fintype.sum_prod_type]

theorem early_prefixReward (a : Bool) :
    (early a).prefixReward unitPrior = if a then -1 else 0 := by
  cases a <;>
    simp [prefixReward, early, unitPrior, Fintype.sum_prod_type]

theorem late_normalised (sel : Boundary → Bool) : Kernel.Normalised (lateSel sel).K := by
  intro s
  simp [lateSel]

theorem lateSel_eu (sel : Boundary → Bool) (a : Bool) :
    (lateSel sel).eu (boundaryLaw a) zeroCont =
      (if a then 4 else 0) + ((if sel (true, a) = true then 1 else 0) +
        (if sel (false, a) = false then 1 else 0)) := by
  simp only [eu, lateSel, boundaryLaw, pushforward, early, unitPrior, zeroCont,
    Fintype.sum_prod_type, Fintype.sum_bool, Finset.univ_unique, Finset.sum_singleton]
  cases a <;> simp only [Bool.true_eq_false, Bool.false_eq_true, if_true, if_false] <;>
    split_ifs <;> norm_num

theorem late_eu (τ : Bool → Bool) (a : Bool) :
    (late τ).eu (boundaryLaw a) zeroCont = (if a then 4 else 0) + 1 := by
  rw [late, lateSel_eu]
  cases τ a <;> simp

theorem composite_eu (a : Bool) (τ : Bool → Bool) :
    ((early a).comp (late τ)).eu unitPrior zeroCont = 1 + (if a then 3 else 0) := by
  rw [eu_comp (early a) (late τ) (late_normalised _), eu_continuation, early_prefixReward]
  change _ + (late τ).eu (boundaryLaw a) zeroCont = _
  rw [late_eu]
  cases a <;> norm_num
```

The isolated-optimisation obstruction

```
theorem early_isolatedOptimal_iff (a : Bool) :
    IsolatedOptimal early unitPrior zeroCont a ↔ a = false := by
  constructor
  · intro h
    have := h false
    rw [early_eu, early_eu] at this
    cases a
    · rfl
    · norm_num at this
  · rintro rfl b
    rw [early_eu, early_eu]
    cases b <;> norm_num

theorem late_isolatedOptimal (a : Bool) (τ : Bool → Bool) :
    IsolatedOptimal late (boundaryLaw a) zeroCont τ := by
  intro τ'
  rw [late_eu, late_eu]

/-- **Isolated optimisation is not globally sound** (`INF-ISOLATED-OBSTRUCTION`). Every
prefix policy that is optimal on its own, followed by every suffix policy that is optimal
on its own against the boundary law that prefix induces, has composite expected utility
`1`; such pairs exist; another pair attains `4`; and `4` is the joint optimum. -/
theorem isolated_optimisation_fails :
    (∀ (a : Bool) (τ : Bool → Bool), IsolatedOptimal early unitPrior zeroCont a →
        IsolatedOptimal late (boundaryLaw a) zeroCont τ →
          ((early a).comp (late τ)).eu unitPrior zeroCont = 1) ∧
      (IsolatedOptimal early unitPrior zeroCont false ∧
```

```
        IsolatedOptimal late (boundaryLaw false) zeroCont (fun _ ⇒ false)) ∧
      ((early true).comp (late fun _ ⇒ false)).eu unitPrior zeroCont = 4 ∧
      (∀ (a : Bool) (τ : Bool → Bool),
        ((early a).comp (late τ)).eu unitPrior zeroCont ≤ 4) ∧
      (1 : ℝ) < 4 := by
  refine ⟨?_, ⟨(early_isolatedOptimal_iff false).mpr rfl, late_isolatedOptimal _ _⟩, ?_, ?_,
    by norm_num⟩
  · intro a τ ha _
    rw [(early_isolatedOptimal_iff a).mp ha, composite_eu]
    norm_num
  · rw [composite_eu]
    norm_num
  · intro a τ
    rw [composite_eu]
    cases a <;> norm_num
```

The hidden-input obstruction

```
/-- The selector that maximises the suffix's reward separately at each boundary state. -/
def peek : Boundary → Bool := Prod.fst

theorem peek_pointwise_max (sel : Boundary → Bool) (s : Boundary) :
    (lateSel sel).W s () ≤ (lateSel peek).W s () := by
  simp only [lateSel, peek]
  split_ifs <;> norm_num

theorem peek_not_legal : ¬ Legal peek := by
  rintro ⟨τ, h⟩
  have h1 := h (true, false)
  have h2 := h (false, false)
  simp only [peek] at h1 h2
  rw [← h1] at h2
  exact Bool.false_ne_true h2

/-- **Pointwise maximisation at a hidden boundary state is not a legal policy**
(`INF-HIDDEN-OBSTRUCTION`). Against the boundary law of the isolated-optimal prefix, every
legal suffix policy has value `1`, while the selector that maximises at each boundary state
(reading the hidden `H`) has value `2` and is not legal. -/
theorem hidden_input_obstruction :
    (∀ sel, Legal sel → (lateSel sel).eu (boundaryLaw false) zeroCont = 1) ∧
      (∀ sel s, (lateSel sel).W s () ≤ (lateSel peek).W s ()) ∧
      (lateSel peek).eu (boundaryLaw false) zeroCont = 2 ∧
      ¬ Legal peek ∧ (1 : ℝ) < 2 := by
  refine ⟨?_, peek_pointwise_max, ?_, peek_not_legal, by norm_num⟩
  · rintro sel ⟨τ, hτ⟩
    have : sel = fun s ⇒ τ s.2 := funext hτ
    subst this
    exact (late_eu τ false).trans (by norm_num)
  · rw [lateSel_eu]
    simp [peek]
    norm_num

end Obstruction

end OpenInfluence
```